ATIVIDADE PRÁTICA — COMUNICAÇÃO CLIENTE-SERVIDOR

PEDIDO SOCKET

Painel de Pedidos em Tempo Real com WebSocket

Hygor Albert Fernandes Marques

Node.js | Express | WebSocket | MVC | JavaScript

Comércio // Tempo Real // Múltiplos Clientes

O SISTEMA

// visão geral

CONTEXTO	PROBLEMA	SOLUÇÃO
Comércio (lanchonete/ restaurante) Múltiplos operadores simultâneos	Pedidos chegam a qualquer momento Cozinha e caixa precisam ver na mesma hora	Painel Kanban em tempo real WebSocket garante atualização instantânea

ESTRATÉGIA

// por que WebSocket?

Estratégia	Latência	Conexão	Ideal para
HTTP Polling	Alta (N seg)	Nova a cada req	Dados pouco frequentes
Long Polling	Baixa a média	1 por resposta	Notificações assíncronas
WebSocket	**Mínima**	**Persistente**	**Tempo real**

INSTANTÂNEO

Pedido aparece para todos ao mesmo tempo, sem delay

BIDIRECIONAL

Qualquer tela pode enviar ou receber a qualquer momento

EFICIENTE

Uma conexão serve N clientes via broadcast

ARQUITETURA MVC

// organização do código

VIEW

public/index.html

Interface do usuário
HTML + CSS + JS
WebSocket client

CONTROLLER

controllers/pedidoController.js

Recebe mensagens WS
Chama o Model
Faz o broadcast

MODEL

models/pedidoModel.js

Dados em memória
Regras de negócio
criar() / avancarStatus()

routes/pedidoRoutes.js → serve a VIEW via HTTP GET /

FLUXO DE UMA MENSAGEM

// do pedido ao painel

01 **Cliente envia pedido**
`ws.send({ tipo: "NOVO_PEDIDO", ... })`

02 **Controller recebe**
`handleMensagem() → chama pedidoModel.criar()`

03 **Model cria o pedido**
Adiciona ao array em memória, retorna objeto

04 **broadcast() dispara**
Envia "PEDIDO_CRIADO" para TODOS os clientes

05 **View atualiza o painel**
`adicionarCard()` renderiza o card na tela

PEDIDO SOCKET

WebSocket · Node.js · MVC · Tempo Real

Atividade Prática — Comunicação Cliente-Servidor